Assignment_5_Introduction_to_Machine_Learning

2026-10-18

1) The CARET package has a number of built in data sets including a cell body segmentation data set published by Hill, LaPan, Li, and Haney in 2007. This data set consists on 119 imaging measurements on 2019 cells. These measurements were generated using High Content Screening (HCS), a method used to capture images of cells and draw boundaries around cell bodies. These boundaries are then used to aid in identifying cell types. In particular, this data set reports measurements of cells from the SK-BR-3 cell line and indicates whether each of the cells imaged were poorly segmented (PS) or well segmented (WS).

a) Read in the segmentation data set and check for missing data.
b) Split this data into a training and testing data set. The original study split the data using half of the data for training and the other half for testing so let's go ahead and follow their lead. Split the data in half, use one half for training and the other half for testing.
c) Train a random forest repeated 3-fold cross validation model that repeats 5 times. Make sure that "Class" is the target variable. Use all of the columns in your training data as predictors.

```
## a) Read in the segmentation data set and check for missing data.

# read in libraries
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.1
```

```
library(AppliedPredictiveModeling)
library(scales)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v lubridate 1.9.5     v tibble    3.3.1
## v purrr     1.2.2     v tidyr     1.3.2
```

```
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x readr::col_factor() masks scales::col_factor()
## x purrr::discard()    masks scales::discard()
## x tidyr::expand()     masks Matrix::expand()
## x dplyr::filter()     masks stats::filter()
## x dplyr::lag()        masks stats::lag()
## x purrr::lift()       masks caret::lift()
## x tidyr::pack()       masks Matrix::pack()
## x tidyr::unpack()     masks Matrix::unpack()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
# read in data
data("segmentationData")

## Check for missing values
sum(is.na(segmentationData)) # no missing data so no imputation needed
```

```
## [1] 0
```

```r
## b) Split this data into a training and testing data set. The original study split the data using hal

# This code splits our data
trainIndex <- createDataPartition(segmentationData$Class, p = 1/2, list = FALSE, times = 1) #split into

# assign 1/2 of data to a training set variable
Train <- segmentationData[ trainIndex,]

# assign remaining 1/2 of data to a testing set variable
Test <- segmentationData[-trainIndex,]

# check dimensions
dim(Train)
```

```
## [1] 1010   61
```

```r
dim(Test)
```

```
## [1] 1009   61
```

```r
## c) Train a random forest repeated 3-fold cross validation model that repeats 5 times. Make sure that

# pre process - center and scale, no imputation needed
preProcValues <- preProcess(Train, method = c("center", "scale"))

# process data and clean
Train_clean <- predict(preProcValues, Train)
Test_clean  <- predict(preProcValues, Test)

# set parameters for cross validation
fitControl <- trainControl(method = "repeatedcv", number = 3, repeats = 5)

# set seed
```

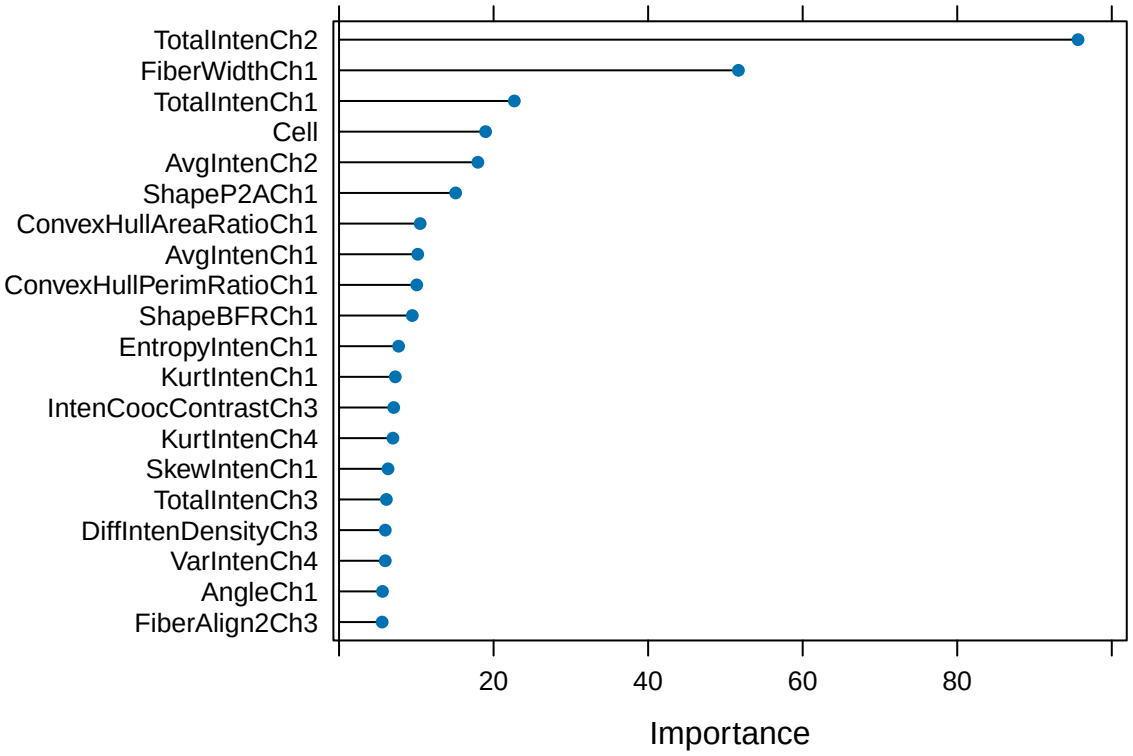
```r
set.seed(123)

# model fitting
Fit1_rf <- train(Class ~ ., # train against all other variables
                 data = Train_clean,
                 method = "rf",
                 trControl = fitControl,
                 verbose = FALSE)
```

2) In the previous question I asked you to train a random forest model on all of the variables in your data set. We do not necessarily require feature selection with random forest models because random forest models inherently preform a type of feature selection. However,feature selection can help us eliminate noisy or redundant features, thereby reducing computational burden. Run the same model again, but this time only include the top 10 variables as predictors.

```r
# find most important variables from the random forest fit
top_variables <- varImp(Fit1_rf, scale = FALSE)

# Make a plot of top twenty variables
plot(top_variables, top = 20)
```




```r
# Order variables by importance, descending, and take the top 10
top10 <- top_variables$importance |>
  arrange(desc(Overall)) |>
```

```
  head(10)

# pull the variables from the top 10
selected_vars <- rownames(top10)
selected_vars <- unique(selected_vars)

# Subset the training set to contain only those variables
featDF <- Train_clean[,selected_vars]

# set parameters
fitControl <- trainControl(method = "repeatedcv", number = 3, repeats = 5)

# Conduct model fitting using x (predictors) and y (outcome)
Fit_rf_feature <- train(x = featDF, # set predictors and outcome
              y = Train_clean$Class,
              method = "rf",
              trControl = fitControl)
```

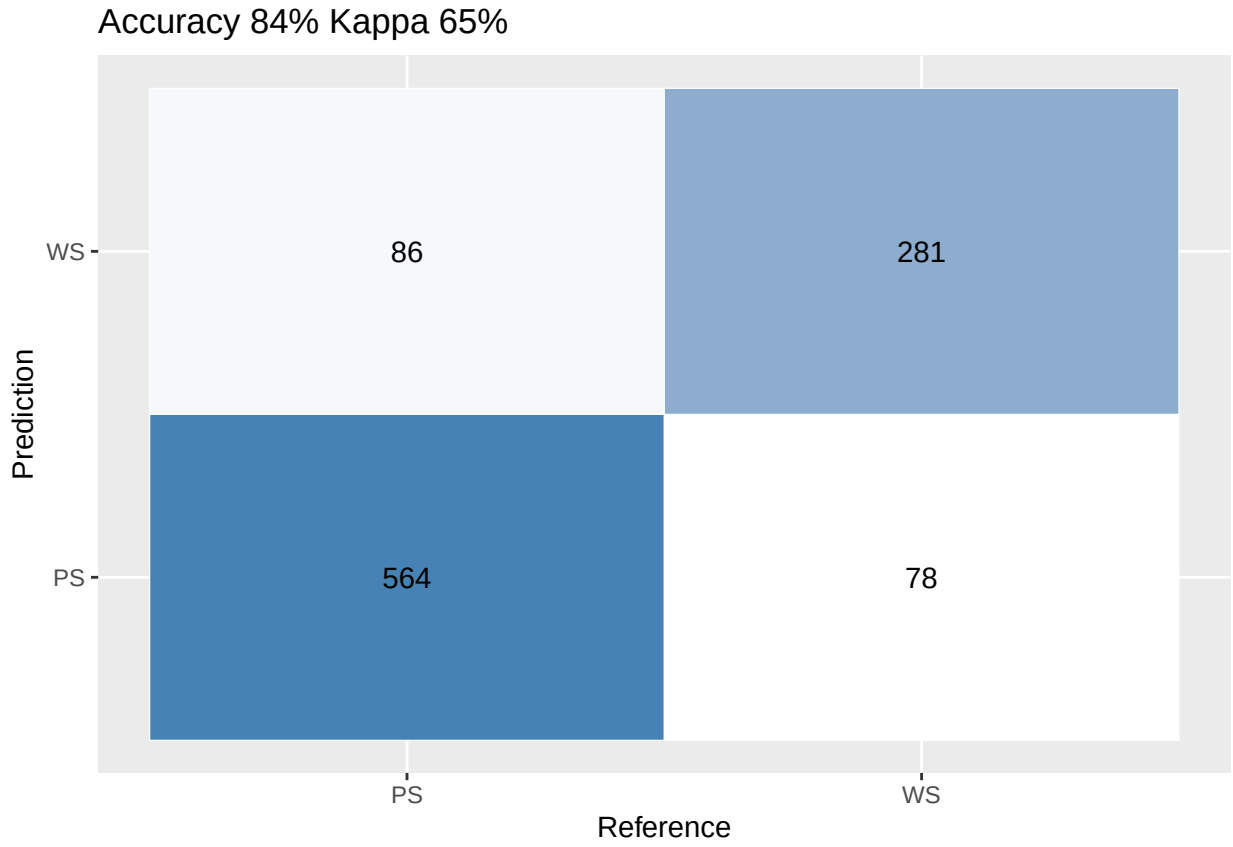
3) Using the model you trained above, generate predictions on the testing set. After running your model, create a confusion matrix visualization in ggplot2. Report the accuracy of your predictions in the main title of your plot. Make sure you have included appropriate x- and y-axis titles! Write your confusion matrix out to a .png file and upload it with your assignment submission. Hint: Here is some starter code to learn how to generate a visualization like this: https://www.energycode.org/posts/09032022-confusion-matrix/.

```
Test_reduced <- Test_clean[, selected_vars] # reduce the test set to only top 10 variables for testing

# Predict and Validate
preds_reduced <- predict(Fit_rf_feature, Test_reduced) # predict based on testing parameters and featur
plot_cfm <- confusionMatrix(preds_reduced, Test_clean$Class) # generate a confusion matrix based on the

# Code sourced from (https://www.energycode.org/posts/09032022-confusion-matrix/)
ggplot_confusion_matrix <- function(cfm) { # function pulled from reference code
  mytitle <- paste("Accuracy", percent_format() (cfm$overall[1]), # setting title with model metrics
                   "Kappa", percent_format() (cfm$overall[2]))
  p <- # plotting table
    ggplot(data = as.data.frame(cfm$table),
           aes(x = Reference, y = Prediction)) +
    geom_tile(aes(fill = log(Freq)), colour = "white") +
    scale_fill_gradient(low = "white", high = "steelblue") + # setting style of fill
    geom_text(aes(x = Reference, y = Prediction, label = Freq)) +
    theme(legend.position = "none") + # setting theme
    ggtitle(mytitle) # add title
  return(p)
}


ggplot_confusion_matrix(plot_cfm) # output plot
```



4) Train a support vector machine model with 3-fold cross validation. Make sure that "Class" is the target variable. Use all of the columns in your training data as predictors. Make sure to indicate that your metric is ROC, as this is a binary classification. Since SVMs are sensitive to the scale of features include "center" and "scale" in your `preProcess` parameter when you train your model.

```
## Note: Data is already pre processed
# reset the fit control for the SVM
# classProbs = TRUE helps generate the probabilties used to calculate ROC
# summaryFunction = twoClassSummary tells caret to calculate ROC AUC
fitControl_svm <- trainControl(method = "repeatedcv", number = 3, repeats = 5,
                               classProbs = TRUE, summaryFunction = twoClassSummary)

# set seed
set.seed(123)

# model fitting
Fit1_svm <- train(Class ~ ., # run svm with class outcome vs. all predictor variables
              data = Train_clean,
              method = "svmLinear", # set the SVM as the model training method
              trControl = fitControl_svm, # use the new fit control I set
              metric = "ROC", # tell the SVM to generate the ROC metric
              verbose = FALSE)

print(Fit1_svm$results) # print out ROC
```

```
##   C       ROC      Sens      Spec      ROCSD     SensSD     SpecSD
```

```
## 1 1 0.8740079 0.8827673 0.6511111 0.01858437 0.02125348 0.05615549
```

5) Generate predictions for your support vector machine model. Like we did with the random forest model, make a confusion matrix visualization. Save the visualization to a .png file and submit it along with this assignment. Make sure to include an appropriate main title with the model accuracy as well as any x- and y- axis titles you think are important!

```
svm_predictions <- predict( # run the predictions using the SVM model
  Fit1_svm,
  Test_clean # use clean test data
)

svm_cm <- confusionMatrix(
  data = svm_predictions, # plot confusion matrix using SVM prediction
  reference = Test_clean$Class # use clean test outcome data to compare against
)

# pull out accuracy
svm_accuracy <- as.numeric(
  svm_cm$overall["Accuracy"]
)

png("svm_confusion_matrix.png", width = 7, height = 8, units = "in", res = 300) # set output for png

svm_cm_title <- paste("Accuracy:", percent_format() (svm_accuracy)) # set title

# reformat above function so it works for SVM model
ggplot(data = as.data.frame(svm_cm$table),
           aes(x = Reference, y = Prediction)) +
  geom_tile(aes(fill = log(Freq)), colour = "white") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  geom_text(aes(x = Reference, y = Prediction, label = Freq)) +
  theme(legend.position = "none") +
  ggtitle(svm_cm_title)

dev.off() # output
```

```
## pdf
##   2
```